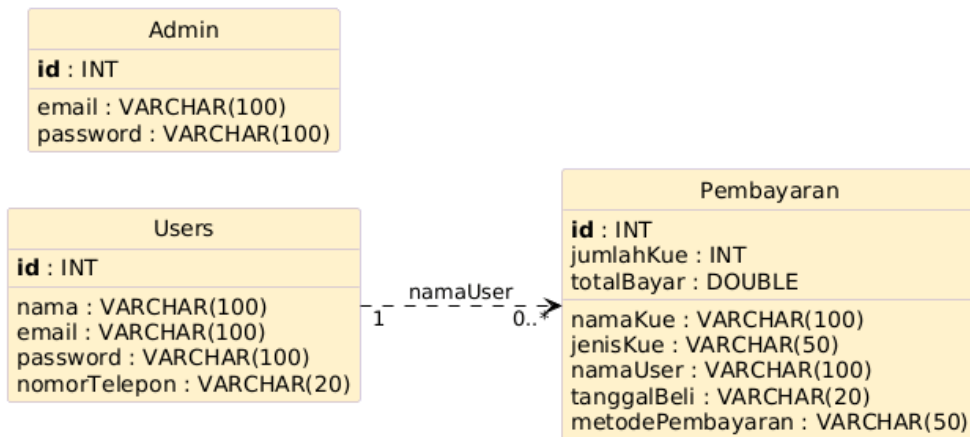


Entity Relationship Diagram (ERD) & Struktur Data

Please use '!option handwritten true' to enable handwritten



Sistem Basis Data Toko Kue Jihan dirancang menggunakan pendekatan relasional modular untuk menjamin efisiensi penyimpanan lokal H2, menjaga kecepatan respons pembaruan stok kue secara *real-time*, serta mengeliminasi risiko anomali data transaksi. Di bawah ini adalah pemisahan komponen basis data, struktur atribut, ragam tipe data, hingga aturan kardinalitas yang berlaku pada sistem.

Spesifikasi Atribut Entitas

Sistem ini didukung oleh 3 entitas utama yang saling terintegrasi secara *loose coupling*:

A. Entitas USERS (Akun Pelanggan) Menyimpan informasi identitas seluruh pelanggan yang melakukan registrasi dan transaksi pemesanan pada sistem Toko Kue Jihan.

- **id (INT - Primary Key):** Pengidentifikasi unik untuk setiap akun pelanggan. Nilai id digenerate secara otomatis oleh database H2 menggunakan strategi *auto-increment* melalui anotasi `@GeneratedValue(strategy = GenerationType.IDENTITY)` pada *entity class* Java, sehingga setiap pelanggan baru mendapatkan ID unik tanpa perlu diisi manual.
- **nama (VARCHAR/100):** Nama lengkap pelanggan yang digunakan untuk keperluan personalisasi antarmuka kasir dan ditampilkan pada nota pesanan kue sebagai identitas pembeli.
- **email (VARCHAR/100):** Alamat email aktif pelanggan yang berfungsi sebagai *username* saat proses autentikasi login ke sistem toko.
- **password (VARCHAR/100):** Kata sandi keamanan akun pelanggan yang disimpan dalam database untuk verifikasi identitas pada modul login.
- **nomorTelepon (VARCHAR/20):** Nomor kontak aktif pelanggan. Di sisi *frontend*, field ini dibatasi untuk menerima karakter angka dengan panjang maksimum 13 karakter demi validasi pengiriman logistik kue.

B. Entitas PEMBAYARAN (Transaksi & Kasir Logistik Kue) Merupakan entitas transaksi utama (*core transaction entity*) yang merekam seluruh manifestasi belanja dan rekapitulasi nominal pembelian kue oleh pelanggan.

- **id (INT - Primary Key):** Pengidentifikasi unik setiap nomor urut transaksi kasir yang digenerate otomatis oleh database H2 menggunakan *auto-increment*.
- **namaKue (VARCHAR/100):** Nama varian produk kue yang dibeli langsung dari katalog (misalnya: Kue Tart, Croissant, Brownies).
- **jenisKue (VARCHAR/50):** Kategori atau jenis klasifikasi kue yang dipesan untuk mempermudah pemilahan logistik produksi di dapur toko.
- **namaUser (VARCHAR/100):** Nama pelanggan yang melakukan transaksi. Diambil dari representasi teks data sesi pengguna aktif dan disimpan langsung untuk kebutuhan filter histori transaksi tanpa kueri *join*.
- **tanggalBeli (VARCHAR/20):** Catatan waktu riil eksekusi pesanan oleh backend Spring Boot yang disimpan sebagai string untuk kemudahan format tanggal lokal.
- **jumlahKue (INT):** Kuantitas atau jumlah item kue yang diborong pelanggan dalam satu transaksi, memotong ketersediaan inventaris pada tabel PRODUK.
- **metodePembayaran (VARCHAR/50):** Pilihan penyelesaian tagihan yang digunakan oleh pelanggan (seperti Tunai atau QRIS otomatis).
- **totalBayar (DOUBLE):** Total nominal belanja akhir yang wajib dibayarkan, bertipe DOUBLE untuk memfasilitasi perhitungan matematis desimal presisi.

C. Entitas ADMIN (Pengelola Toko) Menyimpan kredensial hak akses kelompok pengelola (Bintang, Arif, Rafly). Entitas ini berdiri mandiri (*standalone*) tanpa memiliki batasan kunci asing fisik (*foreign key*) ke entitas lain di level skema database guna mengisolasi keamanan kredensial. Interaksi pengelolaan dilakukan penuh di level aplikasi via REST API.

- **id (INT - Primary Key):** Nomor unik akun administrator yang digenerate otomatis oleh database lokal H2.
- **email (VARCHAR/100):** Alamat email resmi pengelola yang digunakan untuk masuk ke *dashboard* internal toko.
- **password (VARCHAR/100):** Kata sandi admin yang dicocokkan secara sensitif (*exact-match*) menggunakan custom query method `findByEmailAndPassword()` pada komponen AdminRepository melalui perintah internal `SELECT * FROM ADMIN WHERE email = ? AND password = ?`

Analisis Kardinalitas dan Aturan Bisnis Relasi (*Business Rules*)

Berdasarkan visualisasi skema data, arsitektur Toko Kue Jihan menegakkan aturan operasional berikut:

1. Relasi USERS — PEMBAYARAN (One-to-Many / 1:N — Non-Enforced Relationship)

- Sisi Hubungan: Tabel USERS berperan sebagai entitas induk (*parent*) dan PEMBAYARAN sebagai entitas anak (*child*).
- Aturan Kardinalitas: Satu pelanggan dapat melakukan banyak (Many) kali pemesanan dan transaksi belanja kue dari waktu ke waktu. Namun, satu rekam data baris PEMBAYARAN hanya boleh berelasi secara sah dengan satu (One) nama pelanggan yang diidentifikasi lewat string atribut `namaUser`.
- Implikasi Desain: Hubungan ini digambarkan dengan garis putus-putus (*weak association*) karena tidak mengikat *Foreign Key* fisik di database H2. Hal ini sengaja diimplementasikan untuk mengeliminasi beban kueri *join* yang berat, mempercepat proses simpan transaksi kasir, serta mencegah runtutan eror (*cascade error*) jika ada perubahan data master pengguna.

2. Relasi ADMIN — USERS & PEMBAYARAN (Standalone / Isolasi Penuh)

- Sisi Hubungan: Tabel ADMIN terisolasi penuh dari siklus data tabel pelanggan maupun transaksi belanja.
- Aturan Hubungan Fungsional: Tidak ada aturan kardinalitas fisik di level database. Rantai kendali admin terhadap data master toko murni dijembatani secara asinkronus lewat REST API di layer Controller (`AdminRestController` -> `UserService` / `ProdukService` -> H2 Database). Alasan pemisahan ini agar modul autentikasi admin kelompok 4 tetap ringan, independen, dan aman dari manipulasi injeksi data luar.

3. Desain Integritas Referensial Database

Untuk menjamin konsistensi data logistik toko kue terlindungi dengan baik, arsitektur data menerapkan prinsip berikut:

- Auto-Increment Primary Keys: Pencegahan duplikasi nomor transaksi atau ID dilakukan dengan menyematkan strategi `GenerationType.IDENTITY` pada seluruh aspek kunci utama tabel (USERS, PEMBAYARAN, ADMIN).
- Entity Mapping berbasis JPA: Melalui anotasi `@Entity` dan `@Table` pada *source code* Spring Boot, Hibernate secara otomatis membangun skema tabel lokal sesaat setelah server aktif, mengeliminasi proses penulisan skrip SQL manual oleh anggota kelompok.
- Data Normalization: Skema database dirancang bersih dan modular tanpa adanya redundansi data yang tidak perlu di antara ketiga entitas utama tersebut.